SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Course Code:	**CSA1202**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO5**	**Assessment:**	**ASSESSMENT TOOL2- PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)**
Weightage:	**30%**	**Total Marks:**	**25**
CO5 Statement:	*Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.*		
Student Name:	**GAYATHERI S**	**Reg.No.:**	**192521306**
Department:	**IT**	**Date:**	**30.08.2026**

ANNEXURE A
Pseudocode Writing Task (Algorithm Design)

1. Write pseudocode to design a DMA-based data transfer algorithm that moves data from an I/O device to memory while handling interrupts and minimizing CPU involvement.
2. Develop pseudocode for an interrupt handling system that prioritizes vectored interrupts from multiple devices and dispatches appropriate service routines.
3. Write pseudocode to select between memory-mapped I/O and I/O-mapped I/O dynamically based on latency and bandwidth requirements.
4. Design pseudocode for a bus arbitration algorithm that manages multiple devices communicating over PCIe, USB, and Thunderbolt interfaces.
5. Write pseudocode to implement a hybrid I/O communication mechanism that uses polling for low-speed devices and DMA with interrupts for high-speed devices.

Code of Conduct:

I GAYATHERI S (192521306) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
GAYATHERI S

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs

Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. DMA-Based Data Transfer Algorithm Problem

Design an algorithm that transfers data directly from an **I/O device to memory using DMA**, while minimizing CPU involvement and properly handling interrupts.

Inputs

- Device_ID – source I/O device
- Memory_Address – starting memory location
- Data_Size – number of bytes/words to transfer
- Transfer_Mode – read/write mode
- DMA_Channel

Outputs

- Data successfully transferred to memory
- Transfer completion/error status
- Interrupt generated after completion

Constraints

- CPU should not handle every individual data transfer.
- DMA controller must control the transfer.
- CPU should be interrupted only when necessary.
- Memory and device addresses must be valid.
- Errors such as device failure or DMA failure must be handled.

Pseudocode

ALGORITHM DMA_DATA_TRANSFER

```
INPUT:
   Device_ID
   Memory_Address
   Data_Size
   Transfer_Mode
   DMA_Channel

OUTPUT:
   Transfer_Status

BEGIN

   IF Device_ID is invalid THEN
      RETURN "ERROR: Invalid I/O Device"
   END IF

   IF Memory_Address is invalid THEN
      RETURN "ERROR: Invalid Memory Address"
   END IF

   IF Data_Size <= 0 THEN
      RETURN "ERROR: Invalid Data Size"
   END IF

   // Step 1: Initialize DMA controller
   DMA.Initialize(DMA_Channel)

   // Step 2: Configure DMA registers
   DMA.Source ← Device_ID
   DMA.Destination ← Memory_Address
   DMA.Transfer_Count ← Data_Size
```

```
DMA.Mode ← Transfer_Mode

// Step 3: Enable DMA interrupt
DMA.Enable_Interrupt()

// Step 4: Request control of system bus
DMA.Request_Bus()

WAIT until DMA receives bus grant

// Step 5: Start direct data transfer     DMA.Start_Transfer()

WHILE DMA.Transfer_Count > 0 DO

   IF Device is ready THEN
      DMA.Transfer_Data()
      DMA.Update_Address()
      DMA.Decrement_Count()
   ELSE
      WAIT for device readiness
   END IF

   IF DMA_Error = TRUE THEN
      DMA.Stop_Transfer()
      DMA.Release_Bus()
      RETURN "ERROR: DMA Transfer Failed"
   END IF

END WHILE

// Step 6: Complete transfer
DMA.Stop_Transfer()
```

```
    // Step 7: Release system bus
    DMA.Release_Bus()

    // Step 8: Generate interrupt to CPU
    DMA.Generate_Interrupt()

    // Step 9: CPU executes interrupt service routine
    ON DMA_INTERRUPT DO
       Read DMA_Status
       IF DMA_Status = SUCCESS THEN          Transfer_Status ← "TRANSFER COMPLETED"
       ELSE
          Transfer_Status ← "TRANSFER FAILED"
       END IF

       Clear DMA_Interrupt()
    END ON

    RETURN Transfer_Status

END
```

Explanation

1. The CPU first validates the device, memory address and transfer size.
2. The CPU configures the DMA controller with the source, destination and transfer count.
3. DMA requests control of the system bus.
4. After receiving bus control, DMA transfers data directly between the I/O device and memory.
5. The CPU is not involved in every byte/word transfer.
6. DMA keeps updating the memory address and decreasing the transfer count.
7. After completion, DMA releases the bus.
8. DMA generates an interrupt.
9. The CPU handles only the completion interrupt.

Key Optimization

CPU involvement is minimized because the DMA controller performs the actual data movement.

---

2. Priority-Based Vectored Interrupt Handling System Problem

Develop an interrupt handling algorithm that receives interrupts from multiple devices, determines their priority, and dispatches the appropriate Interrupt Service Routine (ISR).

Inputs

- Interrupt requests from multiple devices
- Device priority levels
- Interrupt vector table

Outputs

- Correct ISR executed for the highest-priority interrupt
- Interrupt status updated

Constraints

- Multiple devices may request interrupts simultaneously.
- Higher-priority devices must be serviced first. • Lower-priority requests must remain pending.
- Nested interrupts may be allowed depending on priority.
- The system must prevent invalid interrupt vectors.

Pseudocode

```
ALGORITHM PRIORITY_VECTORED_INTERRUPT_HANDLER

INPUT:
   Interrupt_Request[1...N]
   Priority[1...N]
   Interrupt_Vector_Table

OUTPUT:
   Interrupt_Service_Status

BEGIN

   Initialize Interrupt_Controller
   Enable Global_Interrupts
```

```
WHILE System_Running = TRUE DO

   // Step 1: Check all interrupt requests
   Pending_List ← EMPTY

   FOR i ← 1 TO N DO
      IF Interrupt_Request[i] = TRUE THEN
         Add device i to Pending_List
      END IF
   END FOR

   // Step 2: Check whether any interrupt is pending
   IF Pending_List is NOT EMPTY THEN

      // Step 3: Select highest-priority interrupt
      Highest_Priority_Device ←
         Find_Highest_Priority(Pending_List, Priority)

      // Step 4: Obtain interrupt vector
      Vector ← Interrupt_Vector_Table[
                 Highest_Priority_Device
               ]

      IF Vector is INVALID THEN
         Record "Invalid Interrupt Vector"
         Clear Interrupt_Request[
            Highest_Priority_Device
         ]
         CONTINUE
      END IF
```

```
// Step 5: Save CPU context
Save CPU_Registers
Save Program_Counter
Save Processor_Status

// Step 6: Disable or mask lower/equal priority interrupts
Mask_Lower_Priority_Interrupts(
   Priority[Highest_Priority_Device]
)

// Step 7: Acknowledge interrupt
Send_Interrupt_Acknowledge(
   Highest_Priority_Device
)

// Step 8: Dispatch appropriate ISR
CALL ISR[Vector]

// Step 9: Clear serviced interrupt
Interrupt_Request[
   Highest_Priority_Device
] ← FALSE

// Step 10: Restore CPU context
Restore Processor_Status
Restore Program_Counter
Restore CPU_Registers

// Step 11: Re-enable interrupts
Unmask_Interrupts()

Return_From_Interrupt()
```

```
    END IF

  END WHILE

END
```

Function: Find Highest Priority

```
FUNCTION Find_Highest_Priority(Pending_List, Priority) BEGIN

  Highest ← Pending_List[1]

  FOR each Device in Pending_List DO

    IF Priority[Device] > Priority[Highest] THEN
      Highest ← Device
    END IF

  END FOR

  RETURN Highest

END FUNCTION
```

Example

Suppose four devices generate interrupts:

Device	Interrupt	Priority
Keyboard	INT1	2
Network	INT2	4
Disk	INT3	3
Timer	INT4	5

If all four interrupts occur together:

Timer → Priority 5 → serviced first

Network → Priority 4 → serviced second

Disk → Priority 3 → serviced third

Keyboard → Priority 2 → serviced last

Key Features

- Uses **priority comparison**.
- Uses **vectored interrupts**.
- Uses a **vector table** to identify the correct ISR.

---

3. Dynamic Selection Between Memory-Mapped I/O and I/O-Mapped I/O Problem

Design an algorithm that dynamically selects the appropriate I/O addressing technique based on **latency and bandwidth requirements**.

Inputs

- Latency_Requirement
- Bandwidth_Requirement
- Device_Type
- CPU_Address_Space
- Available_Memory
- I/O_Port_Availability

Outputs

- Selected I/O technique:
    - Memory-Mapped I/O ○
      I/O-Mapped I/O

Basic Principle Memory-Mapped I/O

- I/O registers are treated like memory locations.
- Provides flexible addressing and normal memory instructions.
- Suitable when fast access and high bandwidth are important.

I/O-Mapped I/O

- I/O devices have a separate I/O address space.
- Preserves memory address space.
- Suitable for simpler devices and when memory address space is limited.

Pseudocode

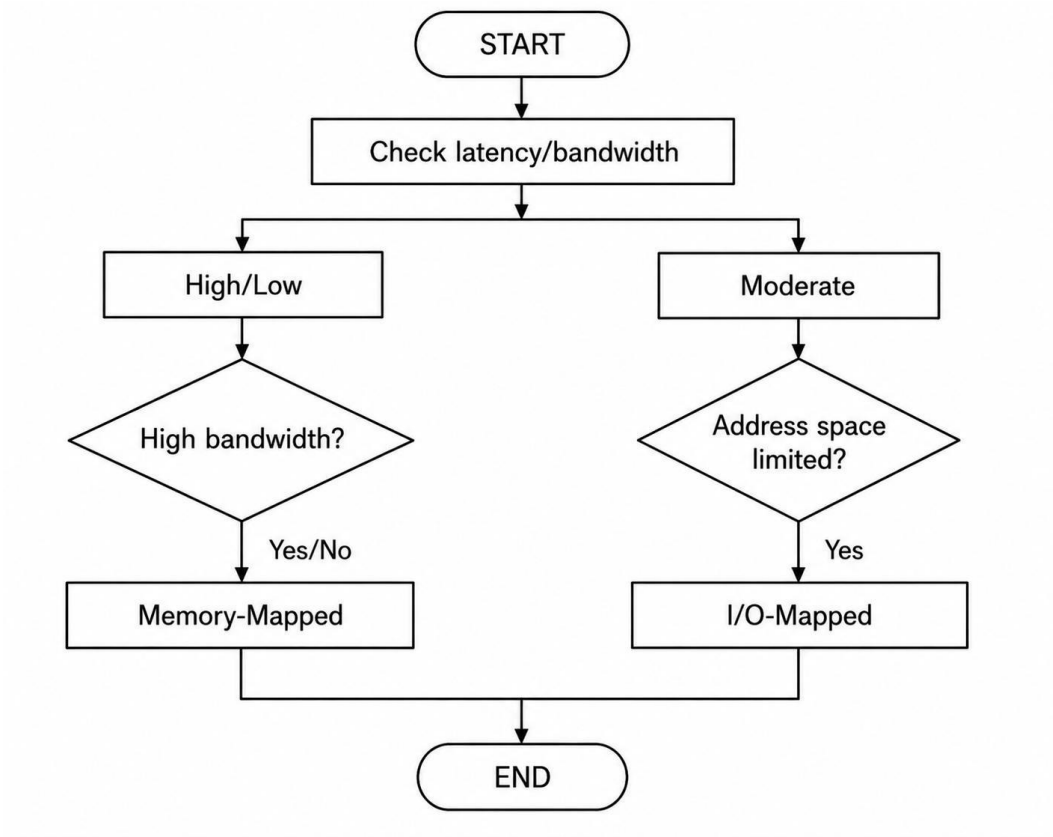
ALGORITHM DYNAMIC_IO_MAPPING_SELECTION

```
INPUT:
   Latency_Requirement
   Bandwidth_Requirement
   Device_Type
   CPU_Address_Space
   Available_Memory
   I/O_Port_Availability

OUTPUT:
   Selected_IO_Method

BEGIN

   // Step 1: Validate requirements

   IF Latency_Requirement <= 0 OR
      Bandwidth_Requirement <= 0 THEN

       RETURN "ERROR: Invalid Requirements"

   END IF

   // Step 2: Check device characteristics

   IF Device_Type = "HIGH_SPEED" THEN

      IF Latency_Requirement = "VERY_LOW" AND
         Bandwidth_Requirement = "HIGH" THEN

         IF Available_Memory = SUFFICIENT THEN
            Selected_IO_Method ← "MEMORY-MAPPED I/O"
         ELSE
```

```
            Selected_IO_Method ← "I/O-MAPPED I/O"
        END IF

    ELSE
        Selected_IO_Method ← "MEMORY-MAPPED I/O"
    END IF

ELSE IF Device_Type = "LOW_SPEED" THEN

    IF I/O_Port_Availability = SUFFICIENT THEN
        Selected_IO_Method ← "I/O-MAPPED I/O"
    ELSE
        Selected_IO_Method ← "MEMORY-MAPPED I/O"
    END IF

ELSE

    // General dynamic decision

    IF Bandwidth_Requirement = "HIGH" OR
       Latency_Requirement = "VERY_LOW" THEN

        Selected_IO_Method ← "MEMORY-MAPPED I/O"

    ELSE IF CPU_Address_Space = "LIMITED" AND
            I/O_Port_Availability = SUFFICIENT THEN

        Selected_IO_Method ← "I/O-MAPPED I/O"

    ELSE

        Selected_IO_Method ← "MEMORY-MAPPED I/O"
```

```
        END IF

    END IF

    // Step 3: Configure selected method

    IF Selected_IO_Method = "MEMORY-MAPPED I/O" THEN
Configure_Device_Using_Memory_Address()
        Use_Normal_Memory_Access_Instructions()

    ELSE
        Configure_Device_Using_IO_Port()
        Use_IO_Specific_Instructions()

    END IF

    RETURN Selected_IO_Method

END
```

Decision Logic



Advantages

Factor	Memory-Mapped I/O	I/O-Mapped I/O
Address space	Uses memory address space	Separate I/O space
Instructions	Normal memory instructions	Special I/O instructions
Flexibility	High	Moderate
High-speed access	Suitable	Suitable for simpler access
Memory conservation	Lower	Better

4. Bus Arbitration Algorithm for PCIe, USB and Thunderbolt Problem

Design an arbitration algorithm that manages multiple devices communicating through **PCIe, USB and Thunderbolt interfaces**.

Inputs

- Device requests
- Interface type
- Priority
- Bandwidth requirement
- Latency requirement
- Device status

Outputs

- Bus/interface access grant
- Transfer scheduling
- Fair resource allocation

Constraints

- High-priority devices should receive timely service.
- Low-priority devices should not starve.
- High-bandwidth devices need sufficient allocation.
- PCIe, USB and Thunderbolt have different characteristics.
- Arbitration should maintain fairness.

Pseudocode

```
ALGORITHM MULTI_INTERFACE_BUS_ARBITRATION

INPUT:
   Device_Request[1...N]
   Interface_Type[1...N]
   Priority[1...N]
   Bandwidth_Requirement[1...N]
   Latency_Requirement[1...N]

OUTPUT:
   Access_Grant[1...N]
```

```
BEGIN

    Initialize Access_Grant[1...N] ← FALSE
    Initialize Waiting_Time[1...N] ← 0

    WHILE System_Running = TRUE DO

        // Step 1: Collect active requests

        Active_Requests ← EMPTY

        FOR i ← 1 TO N DO

            IF Device_Request[i] = TRUE THEN
                Add i to Active_Requests
                Waiting_Time[i] ← Waiting_Time[i] + 1
            END IF

        END FOR

        IF Active_Requests is EMPTY THEN
            CONTINUE
        END IF

        // Step 2: Calculate arbitration score

        FOR each Device in Active_Requests DO

            Score[Device] ←
                Priority[Device] * PRIORITY_WEIGHT
                +
                Bandwidth_Requirement[Device]
```

```
* BANDWIDTH_WEIGHT
            +
            Waiting_Time[Device]
* WAIT_WEIGHT

        IF Latency_Requirement[Device] = "HIGH" THEN
            Score[Device] ←
                Score[Device] + LATENCY_BONUS
        END IF

    END FOR

    // Step 3: Select device with maximum score

    Selected_Device ←
        Find_Maximum_Score(Active_Requests, Score)

    // Step 4: Check interface availability

    Interface ← Interface_Type[Selected_Device]

    IF Interface = "PCIe" THEN

        IF PCIe_Available = TRUE THEN
            Grant_Access(Selected_Device)
            PCIe_Available ← FALSE
        END IF

    ELSE IF Interface = "USB" THEN

        IF USB_Available = TRUE THEN
            Grant_Access(Selected_Device)
            USB_Available ← FALSE
```

```
    END IF

ELSE IF Interface = "Thunderbolt" THEN

    IF Thunderbolt_Available = TRUE THEN
        Grant_Access(Selected_Device)
        Thunderbolt_Available ← FALSE
    END IF

ELSE
    Record "Unsupported Interface"
END IF

// Step 5: Perform data transfer

IF Access_Granted(Selected_Device) = TRUE THEN

    Perform_Transfer(Selected_Device)

    Device_Request[Selected_Device] ← FALSE
    Waiting_Time[Selected_Device] ← 0            Release_Interface(Interface)

END IF

// Step 6: Prevent starvation

FOR each Device in Active_Requests DO

    IF Device ≠ Selected_Device THEN

        IF Waiting_Time[Device] > MAX_WAIT_TIME THEN
            Increase Priority of Device
```

```
            END IF

        END IF

    END FOR

  END WHILE

END
```

Score Calculation

The arbitration score can be represented as:

```
Score =
  Priority × Priority_Weight
  +
  Bandwidth × Bandwidth_Weight
  +
  Waiting_Time × Waiting_Weight
  +
  Latency_Bonus
```

Function

```
FUNCTION Find_Maximum_Score(Request_List, Score)

BEGIN

  Selected ← Request_List[1]

  FOR each Device in Request_List DO

    IF Score[Device] > Score[Selected] THEN
      Selected ← Device
    END IF
```

END FOR

RETURN Selected

END

Interface Handling IF

Interface = PCIe

Allocate PCIe resources

Perform high-speed transfer

ELSE IF Interface = USB

Allocate USB transaction time

Perform transfer

ELSE IF Interface = Thunderbolt

Allocate high-bandwidth channel

Perform transfer

ELSE

Reject request

END IF

Important Features

- Priority-based arbitration
- Bandwidth-aware scheduling
- Latency-aware scheduling
- Waiting-time mechanism
- Starvation prevention
- Handles PCIe, USB and Thunderbolt.
- Uses modular functions for easy extension.
- Efficiently manages multiple simultaneous requests.

---

5. Hybrid I/O Communication Mechanism Problem

Implement a hybrid I/O mechanism that uses:

- **Polling for low-speed devices** • **DMA + interrupts for high-speed devices**

This approach minimizes unnecessary CPU usage while maintaining responsive communication.

Inputs

- Device list
- Device speed
- Data size
- Device status
- Memory address
- Transfer requirements

Outputs

- Successful data transfer
- Device status
- Transfer completion notification

Pseudocode

```
ALGORITHM HYBRID_IO_COMMUNICATION

INPUT:
   Device_List
   Device_Speed
   Data_Size
   Memory_Address

OUTPUT:
   Transfer_Status

BEGIN

   FOR each Device in Device_List DO

      // Step 1: Identify device speed

      IF Device_Speed[Device] = "LOW_SPEED" THEN
```

```
    // ----------------------------
    // POLLING METHOD
    // ----------------------------

    Initialize_Device(Device)

    WHILE Device_Ready(Device) = FALSE DO
        Poll_Device_Status(Device)
    END WHILE

    // Device is ready
    Transfer_Data_By_CPU(Device)

    IF Transfer_Successful(Device) THEN
        Transfer_Status[Device] ← "SUCCESS"
    ELSE
        Transfer_Status[Device] ← "FAILED"
    END IF

ELSE IF Device_Speed[Device] = "HIGH_SPEED" THEN

    // ----------------------------
    // DMA + INTERRUPT METHOD
    // ----------------------------

    Initialize_DMA()

    DMA.Source ← Device
    DMA.Destination ← Memory_Address
    DMA.Transfer_Count ← Data_Size[Device]

    Enable_DMA_Interrupt(Device)
```

```
    DMA.Request_Bus()

    WAIT until Bus_Granted = TRUE

    DMA.Start_Transfer()

    // CPU performs other work
    WHILE DMA_Transfer_Active(Device) = TRUE DO
        CPU.Perform_Other_Task()
    END WHILE

    // Completion interrupt
    ON DMA_INTERRUPT(Device) DO

        IF DMA_Status(Device) = SUCCESS THEN
            Transfer_Status[Device] ← "SUCCESS"
        ELSE
            Transfer_Status[Device] ← "FAILED"
        END IF

        Clear_DMA_Interrupt(Device)

    END ON

ELSE

    Transfer_Status[Device] ←
        "ERROR: Unknown Device Speed"

END IF
```

```
    END FOR

    RETURN Transfer_Status

END
```

Hybrid I/O Decision Function

```
FUNCTION SELECT_IO_METHOD(Device, Data_Size, Device_Speed)

BEGIN

    IF Device_Speed = "LOW_SPEED" THEN

        RETURN "POLLING"

    ELSE IF Device_Speed = "HIGH_SPEED" THEN

        IF Data_Size >= DMA_THRESHOLD THEN
            RETURN "DMA + INTERRUPT"
        ELSE
            RETURN "INTERRUPT-BASED I/O"
        END IF

    ELSE

        RETURN "ERROR"

    END IF

END FUNCTION
```

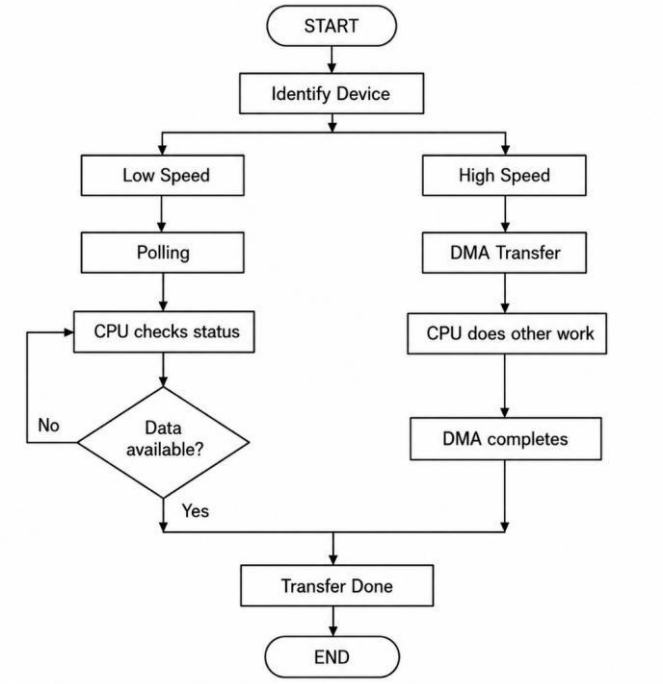
Example

Consider the following devices:

Device	Speed	Data Size	Method

Keyboard	Low	Small	Polling
Mouse	Low	Small	Polling
SSD	High	Large	DMA + Interrupt
Network Adapter	High	Large	DMA + Interrupt
Camera	High	Large	DMA + Interrupt

Working



Comparison of the Five Algorithms

Question	Main Technique	CPU Involvement	Main Control Structure
1	DMA Transfer	Very Low	WHILE, IF, ISR
2	Priority Interrupt	Interrupt-driven	FOR, priority selection, ISR
3	Dynamic I/O Mapping	Requirement-based	IF-ELSE
4	Bus Arbitration	Priority + fairness	FOR, WHILE, scoring
5	Hybrid I/O	Polling + DMA	IF-ELSE, WHILE, ISR